

iTeam University

Tunis --- Tunisia

Projet Semestriel (S4)

Pour l'obtention de :

Ingénieur en Cybersécurité

Thème :

Conception, déploiement et analyse de sécurité d'une application web hébergée sur AWS via la conteneurisation Docker

Élaboré par :

Mohamed Salem KHYARHOUM

Encadré par :

M. Raihane Modhaffer

Année Universitaire : 2025--2026

Dédicaces

À mes parents,

qui m'ont soutenu sans relâche tout au long de mes études et qui n'ont jamais cessé de croire en moi.

À mes frères et sœurs,

pour leurs encouragements constants et leur présence chaleureuse.

À mes enseignants,

qui m'ont transmis le goût du savoir et la rigueur du travail bien fait.

À mes camarades de promotion,

avec qui j'ai partagé tant d'heures de travail, de doutes et de réussites.

À tous ceux qui, de près ou de loin,

*ont contribué à la concrétisation de ce projet,
je dédie ce modeste travail.*

Mohamed Salem

Remerciements

Au terme de ce projet de fin d'études, je tiens à exprimer ma profonde reconnaissance à toutes les personnes qui, par leur soutien, leurs conseils ou leur présence, ont contribué à l'aboutissement de ce travail.

Je remercie tout d'abord **Allah le Tout-Puissant** de m'avoir donné la santé, la patience et la volonté nécessaires pour mener à bien ce projet.

Mes plus sincères remerciements s'adressent à mon encadrant, **M. Raihane Modhaffer**, pour sa disponibilité, ses conseils éclairés et son suivi rigoureux tout au long de la réalisation de ce projet. Ses remarques pertinentes m'ont permis d'orienter mon travail dans la bonne direction et d'en améliorer la qualité.

Je tiens également à remercier l'ensemble du corps enseignant de **l'iTeam University** pour la qualité de la formation reçue durant mes années d'études, ainsi que pour l'environnement académique stimulant qu'ils ont su créer.

Mes remerciements vont aussi aux **membres du jury** qui ont accepté d'évaluer ce travail. Leurs critiques constructives constitueront une précieuse contribution à mon développement professionnel futur.

Enfin, j'adresse une pensée toute particulière à ma **famille** et à mes **amis** pour leur soutien indéfectible, leur patience et leurs encouragements, qui m'ont permis de surmonter les moments difficiles et d'aller jusqu'au bout de cette aventure.

À toutes et à tous, **merci**.

Résumé

Ce projet de fin d'études porte sur la conception, le développement, le déploiement et l'analyse de sécurité d'une application web de prise de notes personnelle, baptisée **WebCyber**. L'application repose sur le *framework* Python **Flask** [1] pour la couche métier, sur **PostgreSQL** pour la persistance des données, et sur **Nginx** comme proxy inverse assurant la terminaison TLS. L'ensemble est **conteneurisé** avec Docker et orchestré via Docker Compose, puis **déployé sur AWS** (instance EC2 t2.micro sous Ubuntu Server 22.04 LTS), avec une résolution de nom de domaine gérée par **Route 53** et un certificat HTTPS automatique fourni par **Let's Encrypt**.

Une attention particulière a été portée à la sécurité, organisée en **six couches** : pare-feu réseau (*Security Group* AWS), durcissement du système, isolation Docker, configuration TLS et en-têtes de sécurité Nginx, contrôles applicatifs (authentification, hachage PBKDF2-SHA256, protection CSRF, validation des entrées, isolation des données par utilisateur) et sécurité de la base de données. La solution déployée a ensuite été soumise à une campagne d'évaluation à l'aide de **Nmap**, **Nikto** et **testssl.sh**, dont les résultats sont analysés et discutés.

Mots-clés : application web, sécurité, cybersécurité, AWS, EC2, Docker, conteneurs, Nginx, Flask, PostgreSQL, HTTPS, TLS, Let's Encrypt, Nmap, Nikto, défense en profondeur, DevOps.

Abstract

This end-of-studies project covers the design, development, deployment and security assessment of a personal note-taking web application named **WebCyber**. The application is built on the Python **Flask** framework for business logic, **PostgreSQL** for data persistence, and **Nginx** as a reverse proxy handling TLS termination. The whole stack is **containerised** with Docker and orchestrated through Docker Compose, then **deployed on AWS** (EC2 t2.micro instance running Ubuntu Server 22.04 LTS), with DNS resolution provided by **Route 53** and HTTPS certificates automatically issued by **Let's Encrypt**.

Security has been treated as a first-class concern and structured into **six layers**: AWS Security Group network firewall, host hardening, Docker isolation, Nginx TLS and security headers, application-level controls (authentication, PBKDF2-SHA256 password hashing, CSRF protection, input validation, per-user data isolation) and database security. The deployed solution was then assessed using **Nmap**, **Nikto** and **testssl.sh**, and the results are analysed and discussed.

Keywords: web application, security, cybersecurity, AWS, EC2, Docker, containers, Nginx, Flask [1], PostgreSQL, HTTPS, TLS, Let's Encrypt, Nmap, Nikto, defense-in-depth, DevOps.

Table des matières

Contexte	12
Problématique	12
Objectifs	12
Plan du rapport	13
1. Introduction	14
2. Cadre institutionnel	14
3. Cloud computing	14
3.1. Définition	14
3.2. Modèles de service	14
3.3. Le fournisseur AWS	15
4. Conteneurisation	15
4.1. Du <i>bare-metal</i> aux conteneurs	15
4.2. Docker et Docker Compose	15
4.3. Bénéfices pour ce projet	15
5. Architecture des applications web modernes	15
5.1. Le modèle 3-tiers	15
5.2. Le rôle du proxy inverse	16
6. Cybersécurité applicative	16
6.1. Le Top 10 OWASP	16
6.2. La défense en profondeur	16
6.3. TLS et HTTPS	16
7. Conclusion du chapitre	17
8. Introduction	18
9. Description générale du projet	18
10. Identification des acteurs	18
11. Besoins fonctionnels	19
11.1. Liste des besoins	19
11.2. Diagramme de cas d'utilisation	19
12. Besoins non fonctionnels	20
12.1. Sécurité	20
12.2. Disponibilité et performance	20
12.3. Déploiement et maintenabilité	20
12.4. Ergonomie	20
12.5. Périmètre exclu	20
13. Conclusion du chapitre	20
14. Introduction	21
15. Architecture globale	21
15.1. Composants et responsabilités	22
16. Infrastructure AWS	22
16.1. Choix de la région	22
16.2. Composants AWS retenus	22
16.3. Règles du Security Group	23
16.4. Estimation des coûts	24
17. Architecture Docker	24

17.1. Vue d'ensemble	24
17.2. Spécifications des conteneurs	25
17.3. Volumes persistants	25
17.4. Estimation des ressources	25
18. Modèle de données	25
18.1. Description	26
18.2. Considérations de sécurité dans le modèle	26
19. Diagrammes de séquence	26
19.1. Authentification	26
19.2. Création d'une note	27
20. Conception de la sécurité — défense en profondeur	28
20.1. Couche 1 — Security Group	29
20.2. Couche 2 — Système hôte	29
20.3. Couche 3 — Isolation Docker	30
20.4. Couche 4 — Nginx + TLS	30
20.5. Couche 5 — Application Flask	30
20.6. Couche 6 — Base de données	30
21. Conclusion du chapitre	30
22. Introduction	31
23. Environnement et outils	31
23.1. Poste de développement	31
23.2. Stack technologique retenue	32
24. Implémentation de l'application Flask	34
24.1. Organisation du code	34
24.2. Modèles SQLAlchemy	34
24.3. Authentification et isolation	34
25. Conteneurisation	34
25.1. Dockerfile applicatif	34
25.2. Fichier docker-compose.yml	34
26. Infrastructure as Code avec Terraform	34
26.1. Motivation	34
26.2. Organisation des fichiers	34
26.3. Bloc Terraform et provider	34
26.4. Ressources principales	34
26.5. Cycle de provisionnement	34
27. Configuration DNS via Route 53	35
28. Préparation automatique de l'instance	35
29. Configuration Nginx et HTTPS	35
29.1. Configuration Nginx	35
29.1.1. Détail des en-têtes	36
29.2. Durcissement des cookies de session Flask	36
29.3. Obtention du certificat Let's Encrypt	36
29.4. Démarrage de la pile	36
30. Pipeline CI/CD avec GitHub Actions	36
30.1. Vue d'ensemble	36
30.2. Extrait du <i>workflow</i>	38

30.3. Secrets GitHub utilisés	38
30.4. Bénéfices de l'approche	38
31. Vue d'ensemble du flux HTTPS	38
31.1. Captures d'écran attendues	39
32. Diagramme de déploiement	39
33. Conclusion du chapitre	40
34. Introduction	41
35. Stratégie de test	42
36. Tests fonctionnels	44
36.1. Inscription et authentification	44
36.2. Gestion des notes (CRUD)	46
36.3. Test critique : isolation des données	48
37. Analyse statique de l'IaC avec tfsec	48
37.1. Principe et positionnement	48
37.2. Exécution	48
37.3. Résultats et décisions	48
37.4. Résultat final	49
38. Vérification de l'infrastructure	49
38.1. Résolution DNS	49
38.2. État des conteneurs	50
38.3. Connectivité	50
38.4. Captures d'écran attendues	50
39. Scan de ports avec Nmap	50
39.1. Objectif	50
39.2. Commandes utilisées	51
39.3. Résultats observés	51
39.4. Recommandations	53
40. Scan applicatif Web avec Nikto	53
40.1. Objectif	53
40.2. Commande	54
40.3. Résultats typiques et interprétation	54
41. Évaluation TLS	54
41.1. SSL Labs	54
41.2. testssl.sh	55
41.3. Résultats	55
42. Synthèse des audits	56
43. Limites de la campagne	56
44. Conclusion du chapitre	57
45. Bilan du projet	58
46. Apports méthodologiques	58
47. Limites et perspectives d'évolution	58
48. Conclusion	59
Bibliographie	60

Liste des figures

Fig. 1	Diagramme de cas d'utilisation de WebCyber.	19
Fig. 2	Architecture globale --- vue d'ensemble.	21
Fig. 3	Infrastructure AWS détaillée.	23
Fig. 4	Architecture Docker --- trois conteneurs sur réseau <code>_bridge_</code>	24
Fig. 5	Diagramme de classes (modèle de données).	26
Fig. 6	Diagramme de séquence --- authentification.	27
Fig. 7	Diagramme de séquence --- création d'une note.	28
Fig. 8	Les six couches de sécurité (<code>_defense-in-depth_</code>).	29
Fig. 9	Flux d'une requête HTTPS, des en-têtes au stockage.	39
Fig. 10	Espace réservé pour une capture d'écran du formulaire de connexion.	39
Fig. 11	Espace réservé pour une capture d'écran de l'interface de notes.	39
Fig. 12	Diagramme de déploiement UML.	40
Fig. 13	Espace réservé pour une capture d'écran de la pile Docker en cours d'exécution.	50

Liste des tableaux

Tableau 1	Tableau des besoins fonctionnels	19
Tableau 2	Tableau des composants et responsabilités	22
Tableau 3	Règles entrantes du Security Group `webcyber-sg`	23
Tableau 4	Estimation mensuelle (sous _free tier_ AWS).	24
Tableau 5	Spécifications des conteneurs	25
Tableau 6	Empreinte mémoire estimée des conteneurs.	25
Tableau 7	Synthèse de la pile technologique.	33
Tableau 8	Enregistrements DNS de la zone hébergée `webcyber.app`	35
Tableau 9	En-têtes de sécurité HTTP appliqués par Nginx.	36
Tableau 10	Étapes du pipeline GitHub Actions.	37
Tableau 11	Secrets configurés dans le dépôt GitHub.	38
Tableau 12	Phases de la campagne de validation.	43
Tableau 13	Tests d'inscription.	44
Tableau 14	Tests d'authentification.	45
Tableau 15	Tests CRUD sur les notes.	47
Tableau 16	Tests d'isolation par utilisateur.	48
Tableau 17	Constats tfsec sur le module Terraform et décisions associées.	49
Tableau 18	Résultats du scan Nmap.	52
Tableau 19	Synthèse des résultats Nikto.	54
Tableau 20	Synthèse de l'évaluation TLS.	55
Tableau 21	Tableau de bord de sécurité.	56

Liste des acronymes

Acronyme	Signification
ACME	<i>Automatic Certificate Management Environment (protocole Let's Encrypt)</i>
AMI	<i>Amazon Machine Image</i>
API	<i>Application Programming Interface</i>
AWS	<i>Amazon Web Services</i>
CI/CD	<i>Continuous Integration / Continuous Deployment</i>
CIDR	<i>Classless Inter-Domain Routing</i>
CRUD	<i>Create, Read, Update, Delete</i>
CSP	<i>Content Security Policy</i>
CSRF	<i>Cross-Site Request Forgery</i>
DNS	<i>Domain Name System</i>
EBS	<i>Elastic Block Store</i>
EC2	<i>Elastic Compute Cloud</i>
EIP	<i>Elastic IP Address</i>
GHA	<i>GitHub Actions</i>
GHCR	<i>GitHub Container Registry</i>
HSTS	<i>HTTP Strict Transport Security</i>
HTTP	<i>Hypertext Transfer Protocol</i>
HTTPS	<i>HTTP Secure</i>
IaaS	<i>Infrastructure as a Service</i>
IaC	<i>Infrastructure as Code</i>
IGW	<i>Internet Gateway</i>
IDOR	<i>Insecure Direct Object Reference</i>
IMDSv2	<i>Instance Metadata Service version 2</i>
IP	<i>Internet Protocol</i>
LTS	<i>Long-Term Support</i>
MVC	<i>Modèle-Vue-Contrôleur</i>
NIST	<i>National Institute of Standards and Technology</i>
ORM	<i>Object-Relational Mapping</i>
OWASP	<i>Open Web Application Security Project</i>
PaaS	<i>Platform as a Service</i>
PBKDF2	<i>Password-Based Key Derivation Function 2</i>
PFE	<i>Projet de Fin d'Études</i>
SaaS	<i>Software as a Service</i>
SAST	<i>Static Application Security Testing</i>

SG	<i>Security Group</i>
SQL	<i>Structured Query Language</i>
SSH	<i>Secure Shell</i>
SSL	<i>Secure Sockets Layer</i>
TCP	<i>Transmission Control Protocol</i>
TLD	<i>Top-Level Domain</i>
TLS	<i>Transport Layer Security</i>
URL	<i>Uniform Resource Locator</i>
UML	<i>Unified Modeling Language</i>
VPC	<i>Virtual Private Cloud</i>
WSGI	<i>Web Server Gateway Interface</i>
XSS	<i>Cross-Site Scripting</i>

Introduction

Contexte

À l'ère de la transformation numérique, la majorité des services quotidiens — communication, gestion documentaire, commerce, administration — repose sur des applications web hébergées dans le *cloud*. Cette omniprésence du Web s'est accompagnée d'une explosion des incidents de sécurité : selon le rapport *Verizon Data Breach Investigations Report 2023*, plus de 80% des compromissions enregistrées exploitent des vulnérabilités applicatives ou des erreurs de configuration. Concevoir une application web ne se résume donc plus à coder un service fonctionnel : il faut également maîtriser son hébergement, son réseau, son cycle de déploiement et sa surface d'attaque.

Le mouvement *DevOps* et l'adoption généralisée des **conteneurs** ont profondément changé la manière de construire et de livrer ces applications. Docker offre une portabilité quasi parfaite entre les environnements de développement et de production, tandis que les fournisseurs de *cloud* comme **Amazon Web Services** (AWS) permettent de provisionner en quelques minutes des infrastructures auparavant réservées aux grands centres de données. La maîtrise de ces outils est désormais une compétence attendue de tout développeur ou administrateur système.

Problématique

Si les briques technologiques sont accessibles, leur *intégration sécurisée* reste un exercice difficile : combien de bases de données exposées sur Internet, combien de certificats expirés, combien d'applications vulnérables aux injections SQL ou au *cross-site scripting* ?

La problématique de ce projet peut être formulée ainsi :

Comment concevoir, développer et déployer une application web fonctionnelle dans le *cloud*, en appliquant de manière explicite et vérifiable les bonnes pratiques de cybersécurité, depuis la couche réseau jusqu'au code applicatif ?

Objectifs

Le projet **WebCyber** apporte une réponse pédagogique et pragmatique à cette problématique en poursuivant les objectifs suivants :

1. Concevoir une application web simple mais réaliste : une application personnelle de prise de notes (CRUD complet, authentification, isolation des données par utilisateur).
2. Conteneuriser l'ensemble des composants (Nginx, application Flask [1], base PostgreSQL) avec Docker et Docker Compose.
3. Déployer la solution sur une infrastructure AWS minimaliste mais représentative (EC2, Elastic IP, Security Group, Route 53).
4. Mettre en œuvre une chaîne de sécurité **en profondeur** couvrant le réseau, le système, les conteneurs, le proxy, le code applicatif et la base de données.
5. Activer HTTPS via un certificat Let's Encrypt, avec renouvellement automatisé.
6. Évaluer la posture de sécurité du déploiement à l'aide d'outils standard (Nmap, Nikto, testssl.sh) et discuter les résultats.
7. Rédiger une documentation technique exploitable permettant à un autre étudiant ou ingénieur de reproduire la solution.

Plan du rapport

Ce mémoire s'articule autour de cinq chapitres :

- Le **chapitre 1** pose le *contexte général* et l'*état de l'art* : *cloud computing*, conteneurisation, architecture des applications web modernes et principales menaces de sécurité (OWASP Top 10).
- Le **chapitre 2** formalise l'*analyse et la spécification des besoins* : acteurs, cas d'utilisation, besoins fonctionnels et non fonctionnels.
- Le **chapitre 3** présente la *conception* de la solution : architecture globale, infrastructure AWS, architecture Docker, modèle de données et conception de la sécurité **en profondeur**.
- Le **chapitre 4** détaille la *réalisation et le déploiement* : choix technologiques, implémentation, construction des images Docker, mise en place de l'instance EC2, configuration de Route 53, du proxy Nginx et des certificats Let's Encrypt.
- Le **chapitre 5** expose la phase de *tests fonctionnels et d'analyse de sécurité* : scans Nmap, audit Nikto, évaluation TLS et synthèse des recommandations.

Une *conclusion générale* dresse le bilan du projet et propose des pistes d'évolution.

Contexte général et état de l'art

1. Introduction

Ce premier chapitre situe le projet dans son contexte académique et technologique. Il rappelle d'abord le cadre institutionnel (l'iTeam University), puis présente les fondements théoriques mobilisés : *cloud computing*, conteneurisation, architecture des applications web modernes et grands principes de cybersécurité. Cette base conceptuelle permet de motiver, dans les chapitres suivants, les choix de conception et d'implémentation retenus.

2. Cadre institutionnel

L'iTeam University est un établissement d'enseignement supérieur mauritanien qui forme depuis plusieurs décennies des cadres dans les domaines de la gestion, de la comptabilité et, plus récemment, du développement informatique. Le présent travail s'inscrit dans le parcours de **Licence en Développement Informatique** et constitue le **Projet de Fin d'Études** (PFE) clôturant ce cursus.

L'objectif pédagogique est double : (i) mettre en pratique de manière intégrée les compétences acquises durant la formation (programmation, bases de données, réseaux, systèmes, sécurité) et (ii) confronter l'étudiant à un scénario réaliste de cycle complet — de la spécification jusqu'au déploiement et à l'audit de sécurité.

3. Cloud computing

3.1. Définition

Le **cloud computing** désigne la mise à disposition à la demande, via Internet, de ressources informatiques (calcul, stockage, réseau, bases de données, services applicatifs). Le NIST en donne une définition de référence reposant sur cinq caractéristiques essentielles [2] :

- libre-service à la demande (*on-demand self-service*);
- accès large via le réseau (*broad network access*);
- mutualisation des ressources (*resource pooling*);
- élasticité rapide (*rapid elasticity*);
- facturation à l'usage (*measured service*).

3.2. Modèles de service

Trois modèles de service principaux sont communément distingués :

- **IaaS** (*Infrastructure as a Service*) : le fournisseur expose des ressources virtualisées brutes (machines, réseaux, disques). Le client gère l'OS, le *middleware* et l'application. *Exemple* : AWS EC2.
- **PaaS** (*Platform as a Service*) : le fournisseur gère l'infrastructure et l'environnement d'exécution. Le client fournit le code applicatif. *Exemple* : AWS Elastic Beanstalk, Heroku.
- **SaaS** (*Software as a Service*) : le client consomme directement une application complète accessible via le Web. *Exemple* : Gmail, Microsoft 365.

Pour ce projet, le modèle retenu est l'**IaaS**, car il offre le meilleur compromis entre maîtrise pédagogique des couches techniques (système, réseau, conteneurs, sécurité) et simplicité de mise en œuvre.

3.3. Le fournisseur AWS

Amazon Web Services est le leader mondial du *cloud* public, avec plus d'une trentaine de *régions* (zones géographiques indépendantes) et une offre couvrant l'ensemble des modèles de service. Les briques utilisées dans ce projet sont :

- **EC2** (*Elastic Compute Cloud*) : instances de machines virtuelles facturées à l'heure ou à la seconde;
- **Elastic IP** : adresse IPv4 publique fixe, associable à une instance EC2;
- **Security Group** : pare-feu *stateful* fonctionnant au niveau de l'instance;
- **Route 53** : service DNS managé (zone hébergée et enregistrement de domaine).

4. Conteneurisation

4.1. Du *bare-metal* aux conteneurs

L'évolution des modèles d'exécution applicatifs peut se résumer en trois grandes étapes :

1. **Bare-metal** : un serveur physique = une application. Coût élevé, ressources sous-utilisées.
2. **Virtualisation** : un serveur physique exécute plusieurs machines virtuelles, chacune contenant son OS complet. Meilleure densité, mais surcharge importante (chaque VM démarre un noyau).
3. **Conteneurs** : plusieurs processus isolés partagent le *noyau* de l'hôte, en utilisant les *namespaces* et les *cgroups* Linux pour cloisonner les ressources et la visibilité. Démarrage en quelques centaines de millisecondes, empreinte mémoire réduite.

4.2. Docker et Docker Compose

Docker est aujourd'hui l'outil de référence pour la création et l'exécution de conteneurs. Il introduit deux concepts fondamentaux :

- l'**image** : un *instantané* immuable contenant l'ensemble des fichiers nécessaires à l'exécution d'une application (binaires, bibliothèques, configuration);
- le **conteneur** : une instance d'exécution d'une image, avec son propre espace de noms (réseau, processus, fichiers).

Docker Compose est un outil d'*orchestration locale* qui permet, à partir d'un unique fichier déclaratif YAML (`docker-compose.yml`), de décrire un ensemble de services, leurs relations, leurs réseaux et leurs volumes. Une seule commande suffit ensuite à démarrer ou arrêter l'ensemble de la pile applicative.

4.3. Bénéfices pour ce projet

- **Reproductibilité** : le même fichier `docker-compose.yml` fonctionne à l'identique sur le poste de développement et sur l'instance EC2.
- **Isolation** : chaque service tourne dans son propre conteneur, avec ses dépendances précises.
- **Sécurité** : le réseau interne Docker permet de n'exposer que le proxy Nginx, en gardant l'application Flask et la base PostgreSQL invisibles depuis Internet.
- **Portabilité** : la migration vers un autre fournisseur *cloud* ne nécessite que de relancer la même pile sur une nouvelle machine.

5. Architecture des applications web modernes

5.1. Le modèle 3-tiers

Une application web suit classiquement un découpage en trois couches :

- **Présentation** (*front-end*) : navigateur du client, qui interprète HTML, CSS et JavaScript;
- **Logique métier** (*back-end*) : serveur applicatif qui traite les requêtes, exécute la logique et rend des réponses;
- **Données** : base de données qui assure la persistance.

5.2. Le rôle du proxy inverse

Un **proxy inverse** est un serveur intermédiaire qui se place entre les clients et l'application. Il assure plusieurs fonctions essentielles :

- **Terminaison TLS** : il déchiffre le trafic HTTPS, ce qui évite à l'application de gérer elle-même les certificats;
- **Routage** : il dirige les requêtes vers le bon *back-end*;
- **Mise en cache** et compression;
- **Sécurité périmétrique** : ajout d'en-têtes, limitation de débit, filtrage.

Pour ce projet, le proxy choisi est **Nginx**, en raison de sa légèreté, de sa robustesse et de la richesse de sa documentation.

6. Cybersécurité applicative

6.1. Le Top 10 OWASP

L'**Open Web Application Security Project** (OWASP) publie périodiquement la liste des dix risques applicatifs les plus critiques. La version 2021, qui sert de référence à ce projet, met notamment l'accent sur [3] :

1. **Broken Access Control** : contournement des contrôles d'accès, notamment via la manipulation d'identifiants (*IDOR*);
2. **Cryptographic Failures** : stockage ou transmission de données sensibles sans chiffrement adéquat;
3. **Injection** : injection SQL, NoSQL, OS command;
4. **Insecure Design** : absence de modélisation de la menace en amont du développement;
5. **Security Misconfiguration** : valeurs par défaut, ports inutiles, en-têtes manquants;
6. **Vulnerable and Outdated Components** : utilisation de bibliothèques non maintenues;
7. **Identification and Authentication Failures**;
8. **Software and Data Integrity Failures**;
9. **Security Logging and Monitoring Failures**;
10. **Server-Side Request Forgery (SSRF)**.

6.2. La défense en profondeur

La **défense en profondeur** (*defense-in-depth*) consiste à empiler plusieurs barrières de sécurité indépendantes, de telle sorte que la compromission d'une couche ne suffise pas à compromettre le système entier. Ce principe, hérité du domaine militaire, est particulièrement pertinent pour les applications hébergées dans le *cloud*, où les surfaces d'attaque sont multiples (réseau, système, conteneurs, code, données).

Le chapitre 3 montrera comment ce principe se traduit concrètement dans WebCyber par six couches de sécurité empilées.

6.3. TLS et HTTPS

Transport Layer Security (TLS) est le protocole standard pour chiffrer les communications sur Internet. **HTTPS** désigne simplement HTTP encapsulé dans TLS. Trois propriétés sont garanties :

- **Confidentialité** : le contenu des échanges ne peut être lu par un tiers;
- **Intégrité** : le contenu ne peut être modifié sans détection;
- **Authenticité** : le client peut vérifier l'identité du serveur grâce à un certificat signé par une autorité de confiance.

Let's Encrypt est une autorité de certification gratuite fondée en 2014 par l'*Internet Security Research Group*, qui émet des certificats TLS valides 90 jours, renouvelables automatiquement via le protocole ACME.

7. Conclusion du chapitre

Ce chapitre a présenté les concepts clés mobilisés tout au long du projet : modèle IaaS et services AWS, conteneurisation par Docker, architecture web 3-tiers, principes OWASP et défense en profondeur, ainsi que TLS et Let's Encrypt. Le chapitre suivant s'appuie sur ces fondations pour formaliser les besoins fonctionnels et non fonctionnels de l'application WebCyber.

Analyse et spécification des besoins

8. Introduction

Ce chapitre formalise ce que l'application doit faire et dans quelles conditions. Il présente successivement les acteurs, les besoins fonctionnels (regroupés en cas d'utilisation), puis les besoins non fonctionnels (sécurité, performance, déploiement, maintenabilité).

9. Description générale du projet

WebCyber est une application web sécurisée de prise de notes personnelle. Chaque utilisateur, après s'être inscrit puis authentifié, peut créer, lire, modifier, archiver, supprimer (corbeille avec restauration) et rechercher des notes textuelles qui lui sont propres. Aucun utilisateur ne peut accéder aux notes d'un autre, même en manipulant directement les URL ou les paramètres de requête.

L'application n'est donc pas une innovation fonctionnelle en elle-même — des solutions comparables (Google Keep, Microsoft OneNote, Apple Notes) existent depuis longtemps. Le choix d'un domaine fonctionnel familier est volontaire : il permet de **concentrer l'effort sur la qualité de la conception, du déploiement et de la sécurité** plutôt que sur la nouveauté de la fonctionnalité.

10. Identification des acteurs

Deux acteurs interagissent avec le système :

- **Visiteur** : tout internaute non authentifié. Il peut accéder uniquement aux pages publiques (page d'accueil, formulaire de connexion, formulaire d'inscription).
- **Utilisateur authentifié** : un visiteur qui s'est connecté avec succès. Il accède à son espace personnel et y gère ses notes.

L'application n'embarque pas d'acteur *Administrateur* : la gestion des utilisateurs (création, suppression d'un compte) reste manuelle, par accès direct à la base de données. Cette simplification est cohérente avec le périmètre académique.

11. Besoins fonctionnels

11.1. Liste des besoins

ID	Besoin	Description
BF-01	S'inscrire	Le visiteur saisit un nom d'utilisateur, un email et un mot de passe; le système crée un compte.
BF-02	Se connecter	L'utilisateur saisit ses identifiants; le système vérifie le hachage et ouvre une session.
BF-03	Se déconnecter	La session est détruite côté serveur; l'utilisateur revient à la page de connexion.
BF-04	Créer une note	L'utilisateur saisit un titre et un contenu; la note est sauvegardée et associée à son compte.
BF-05	Consulter ses notes	Affichage paginé des notes de l'utilisateur, par date de modification décroissante.
BF-06	Voir une note en détail	Affichage du titre et du contenu complet d'une note.
BF-07	Modifier une note	L'utilisateur édite le titre et/ou le contenu; la date de modification est mise à jour.
BF-08	Archiver / désarchiver	La note est marquée is_archived; elle disparaît de la liste principale et apparaît dans l'archive.
BF-09	Mettre à la corbeille	La note est marquée is_trashed; elle est cachée mais peut être restaurée.
BF-10	Restaurer depuis la corbeille	Le drapeau is_trashed est levé; la note revient dans la liste.
BF-11	Supprimer définitivement	Suppression DELETE en base, irrécupérable.
BF-12	Rechercher une note	Filtre par titre ou contenu; résultats limités au compte courant.
BF-13	Isolation par utilisateur	Toute lecture, écriture ou suppression filtre par user_id de la session courante.

Tableau 1. – Tableau des besoins fonctionnels

11.2. Diagramme de cas d'utilisation

La Fig. 1 synthétise les cas d'utilisation et leurs liens avec les acteurs. Les relations « *include* » indiquent que les opérations sur les notes nécessitent une authentification préalable (BF-02).

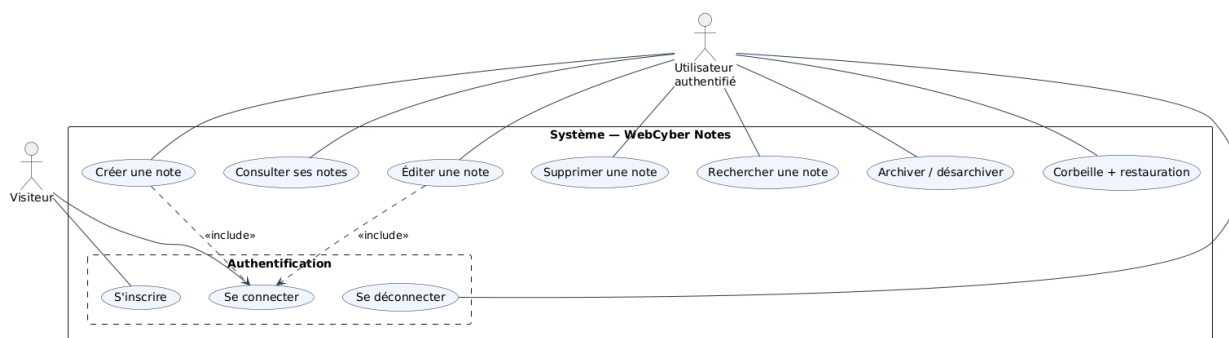


Fig. 1. – Diagramme de cas d'utilisation de WebCyber.

12. Besoins non fonctionnels

Les besoins non fonctionnels traduisent les *qualités attendues* du système, indépendamment des fonctionnalités elles-mêmes.

12.1. Sécurité

- **Confidentialité** : tout le trafic entre le client et le serveur doit être chiffré (HTTPS, TLS 1.2 ou 1.3).
- **Authentification forte** : les mots de passe ne sont jamais stockés en clair; un algorithme de hachage à coût configurable (PBKDF2-SHA256) est utilisé.
- **Protection contre les vulnérabilités OWASP Top 10** : en particulier les injections SQL, le XSS, le CSRF et les contournements de contrôle d'accès.
- **Isolation réseau** : seuls les ports strictement nécessaires (22, 80, 443) sont exposés au public.
- **Isolation des données** : un utilisateur ne peut en aucun cas accéder aux notes d'un autre.

12.2. Disponibilité et performance

- **Disponibilité** : le projet ne vise pas la haute disponibilité; une instance EC2 unique suffit. Le redémarrage automatique des conteneurs (`restart: unless-stopped`) couvre les pannes logicielles.
- **Performance** : la solution doit pouvoir servir une dizaine d'utilisateurs simultanés sur une instance `t2.micro` (1 vCPU, 1 Gio de RAM). Cela correspond largement aux besoins d'une démonstration ou d'un usage personnel.

12.3. Déploiement et maintenabilité

- **Déploiement reproductible** : la solution doit pouvoir être redéployée *from scratch* à partir du dépôt Git, en moins de trente minutes.
- **Conteneurisation complète** : aucun composant n'est installé directement sur l'hôte; tout passe par Docker Compose.
- **Documentation** : l'ensemble des choix d'architecture, de configuration et de sécurité est documenté de manière à permettre à un autre étudiant ou ingénieur de prendre la suite.
- **Renouvellement automatisé** des certificats TLS via Certbot (cron quotidien).

12.4. Ergonomie

- Interface web responsive (mobile et desktop), basée sur **Tailwind CSS** pour un rendu moderne et cohérent.
- Navigation simple : barre latérale pour basculer entre la liste, l'archive et la corbeille; palette de commandes pour un accès rapide.

12.5. Périmètre exclu

Pour rester fidèle au cadre académique du PFE, certains éléments classiques d'une production réelle sont **volontairement exclus** :

- Orchestration Kubernetes ou multi-instance (*auto-scaling*);
- Pipeline CI/CD complet (GitHub Actions + déploiement automatique);
- *Web Application Firewall*, services de protection AWS avancés (WAF, GuardDuty);
- Centralisation des journaux (CloudWatch, ELK);
- Monitoring applicatif (Prometheus, Grafana).

Ces éléments sont mentionnés dans la **conclusion générale** comme perspectives d'évolution.

13. Conclusion du chapitre

L'analyse des besoins fait ressortir un système fonctionnellement simple mais soumis à des exigences non fonctionnelles *nombreuses* (sécurité, déploiement reproductible, isolation, HTTPS). Le chapitre suivant traduit ces exigences en une architecture concrète : choix de l'infrastructure AWS, organisation des conteneurs Docker, modèle de données, conception de la sécurité en profondeur.

Conception

14. Introduction

Ce chapitre traduit les besoins du chapitre précédent en une **architecture technique** précise. Il présente d'abord l'architecture globale du système, puis l'infrastructure AWS et l'architecture Docker, le modèle de données, plusieurs scénarios d'interaction (diagrammes de séquence) et enfin la conception de la sécurité en profondeur.

15. Architecture globale

L'architecture cible peut être résumée par la Fig. 2. Le navigateur du client résout d'abord le nom de domaine `webcyber.app` via Route 53, qui retourne l'**Elastic IP** de l'instance EC2. La requête HTTPS atteint ensuite l'instance, où elle traverse le **Security Group** avant d'être servie par le conteneur Nginx, qui terminera TLS et *proxifiera* la requête vers l'application Flask [1].

Figure 01 — Architecture globale

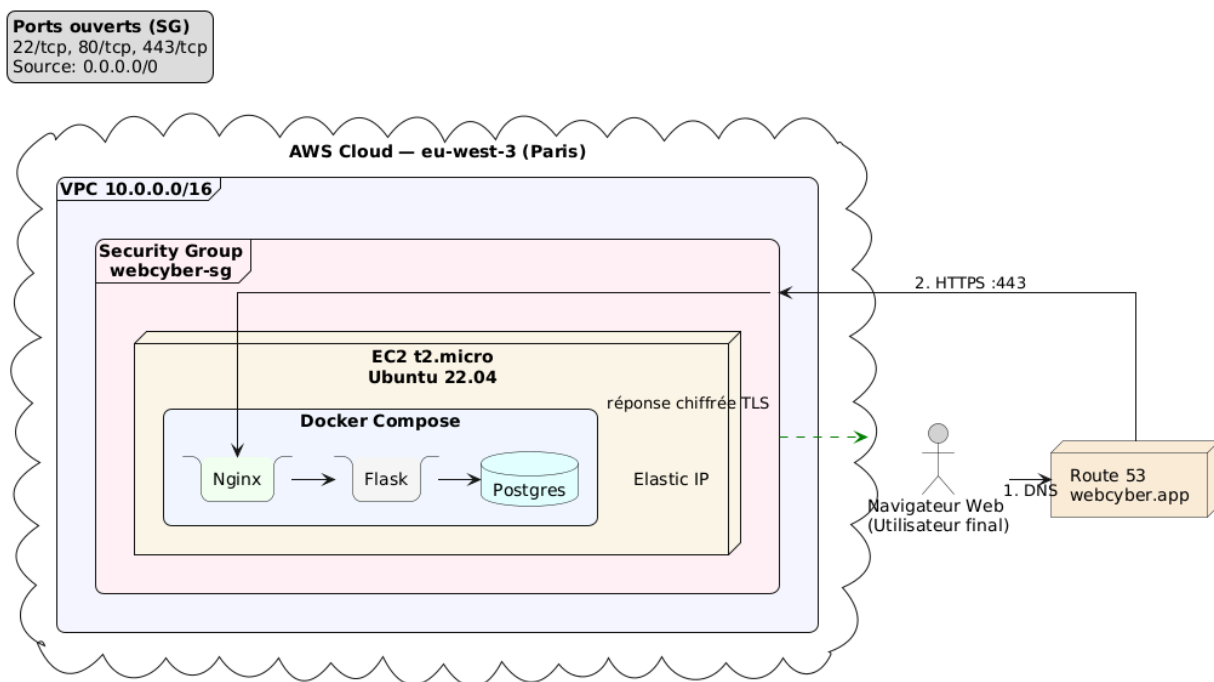


Fig. 2. — Architecture globale --- vue d'ensemble.

15.1. Composants et responsabilités

Composant	Responsabilité
	Route 53
Résolution DNS : webcyber.app → Elastic IP.	Elastic IP
Adresse IPv4 pu- blique fixe asso- ciée à l'EC2.	EC2 (t2.micro)
Hôte Ubuntu 22.04 exécutant Docker.	Security Group
Pare-feu <i>stateful</i> au niveau de l'EC2.	Conteneur Nginx
Proxy inverse, terminaison TLS, en-têtes de sécurité.	Conteneur Flask
Logique applica- tive, authentifi- cation, CRUD.	Conteneur PostgreSQL
Persistance des données utilis- teur.	

Tableau 2. – Tableau des composants et responsabilités

16. Infrastructure AWS

16.1. Choix de la région

La région eu-west-3 (Paris) a été choisie pour sa proximité géographique (latence réduite vers la Mauritanie et l'Europe francophone) et pour des considérations de *résidence des données* (stockage en Europe).

16.2. Composants AWS retenus

La Fig. 3 détaille l'organisation interne au *cloud*. Les choix retenus privilégient la **simplicité** et la **lisibilité** pédagogique :

- utilisation du **VPC par défaut** (CIDR 172.31.0.0/16) sans VPC personnalisé;
- un **sous-réseau public** unique;
- un **Security Group dédié** (webcyber-sg);
- une seule **instance EC2** t2.micro;
- une **Elastic IP** associée;
- une **zone hébergée Route 53** pour le domaine webcyber.app.

Figure 02 — Infrastructure AWS détaillée

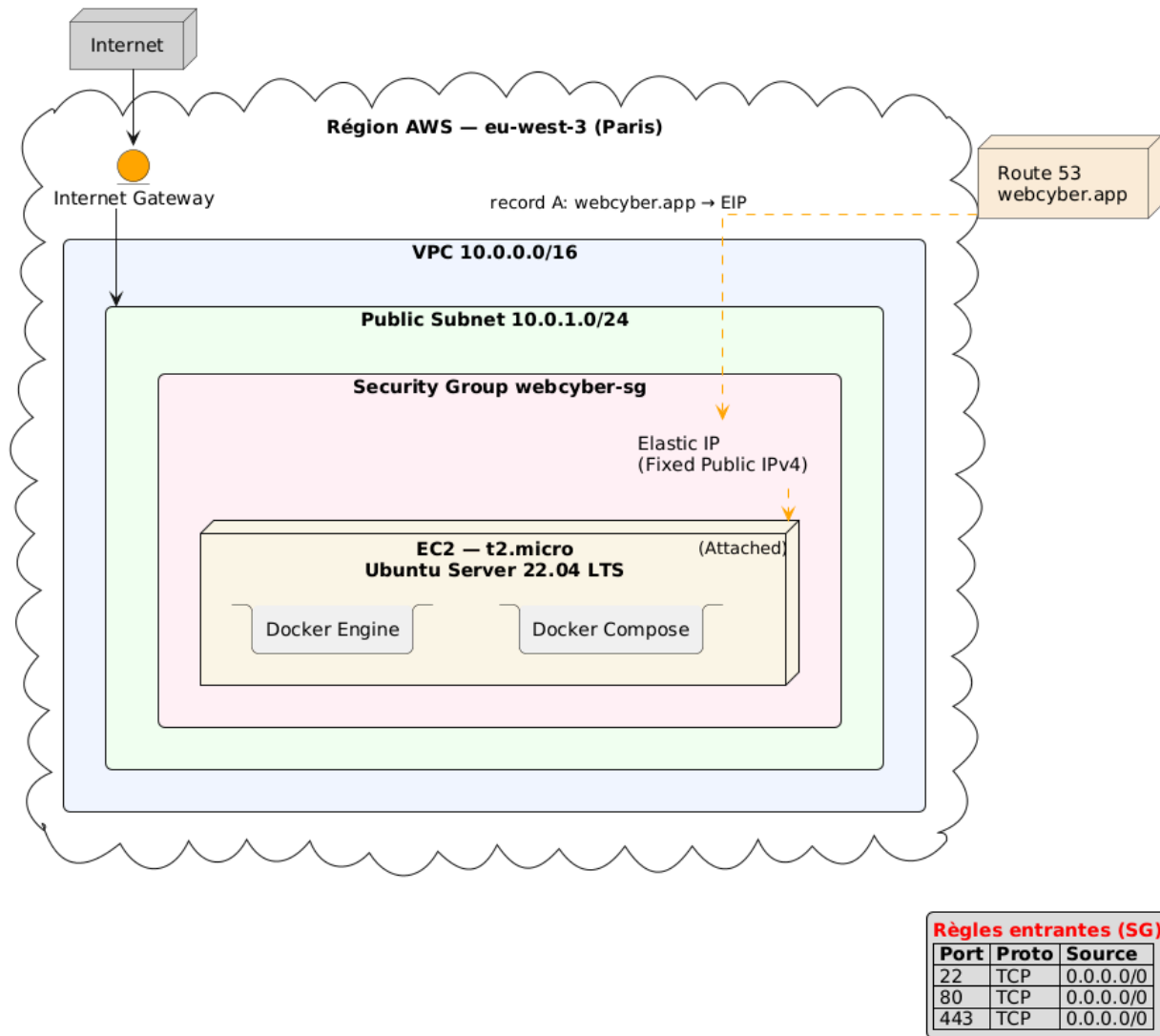


Fig. 3. – Infrastructure AWS détaillée.

16.3. Règles du Security Group

Le pare-feu réseau du périmètre est un **Security Group** *stateful* : seul le trafic entrant explicitement autorisé est accepté, et les réponses correspondantes sont automatiquement autorisées en sortie.

Type	Proto- cole	Port	Source	Justification
Administration distante	SSH	TCP 22	0.0.0.0/0	0.0.0.0/0
Redirection vers HTTPS	HTTP	TCP 80	0.0.0.0/0	0.0.0.0/0
Trafic web chiffré	HTTPS	TCP 443	0.0.0.0/0	0.0.0.0/0

Tableau 3. – Règles entrantes du Security Group `webcyber-sg`.

Aucun autre port n'est ouvert. Les ports applicatifs (5000 pour Flask [1], 5432 pour PostgreSQL) restent confinés au réseau Docker interne et ne sont donc pas accessibles depuis Internet.

16.4. Estimation des coûts

Ressource	Coût
EC2 t2.micro	0,00], [EBS 20 Gio gp3], [0,00
Elastic IP (associée)	0,00], [Route 53 zone hébergée], [0,50
Trafic sortant (faible)	≈ 0,00
Total estimé	≈ 0,50/mois

Tableau 4. – Estimation mensuelle (sous _free tier_ AWS).

17. Architecture Docker

17.1. Vue d'ensemble

Trois conteneurs cohabitent sur l'instance, orchestrés par Docker Compose et reliés par un réseau *bridge* privé (webcyber-net). La Fig. 4 montre cette organisation, ainsi que les volumes persistants utilisés.

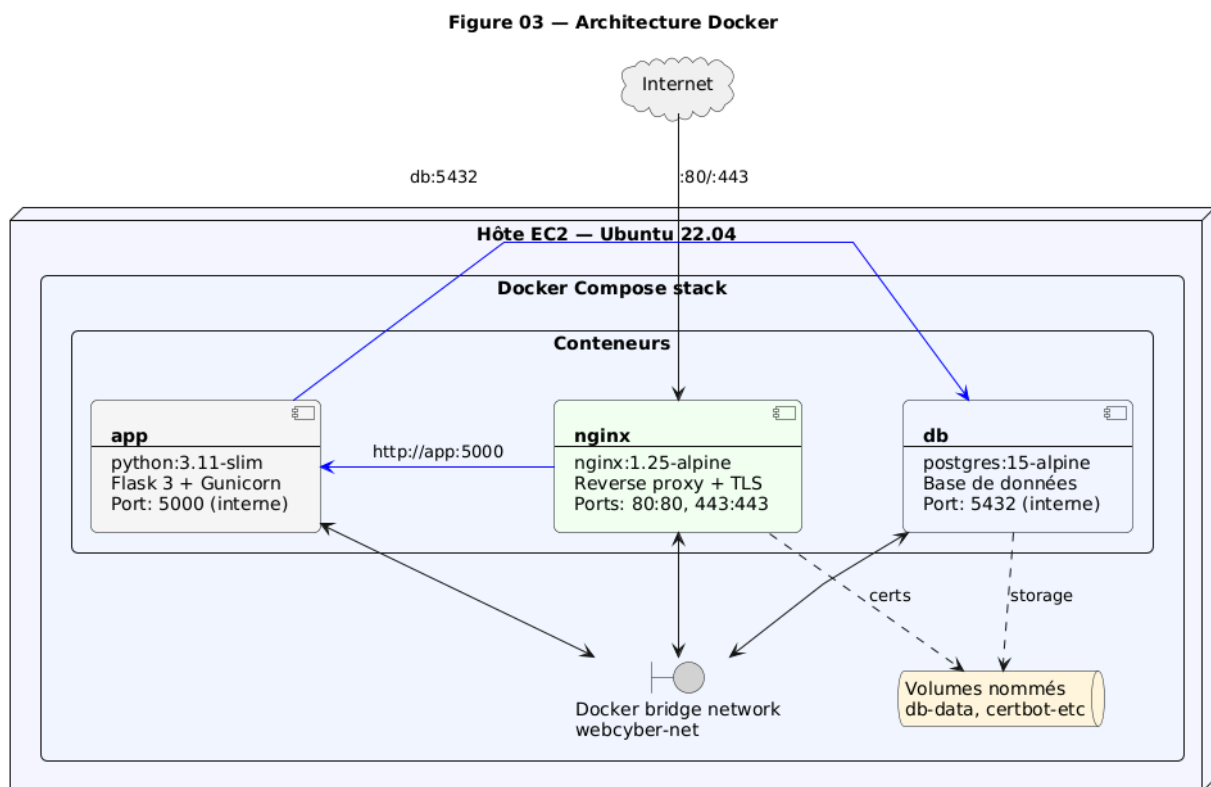


Fig. 4. – Architecture Docker --- trois conteneurs sur réseau _bridge_.

17.2. Spécifications des conteneurs

Service	Image	Ports	Rôle
	nginx	nginx:1.25-alpine	80, 443 (publiés)
Proxy inverse, TLS, entêtes de sécurité	app	python:3.11-slim (build)	5000 (interne)
Application Flask + Gunicorn	db	postgres:15-alpine	5432 (interne)
Base de données PostgreSQL			

Tableau 5. – Spécifications des conteneurs

Le **principe d'exposition minimale** est appliqué : seul le proxy publie ses ports vers l'hôte. Les services app et db ne sont accessibles qu'*intra-Docker*, par leur nom de service (résolution DNS embarquée par Docker).

17.3. Volumes persistants

- db-data : données PostgreSQL (table users, table notes, indexes, WAL).
- certbot-etc : certificats Let's Encrypt et clés privées.
- certbot-var : état interne de Certbot (compte ACME, *lock files*).

17.4. Estimation des ressources

Sur t2.micro (1 Gio RAM), l'occupation mémoire prévisionnelle est confortable :

Conteneur	Mémoire estimée
	Nginx
10–30 Mio	Flask + 2 <i>workers</i> Gunicorn
100–200 Mio	PostgreSQL
50–100 Mio	Surcharge Docker
≈ 100 Mio	
Total	≈ 300–430 Mio

Tableau 6. – Empreinte mémoire estimée des conteneurs.

18. Modèle de données

Le modèle de données est volontairement réduit à deux entités, comme le montre la Fig. 5.

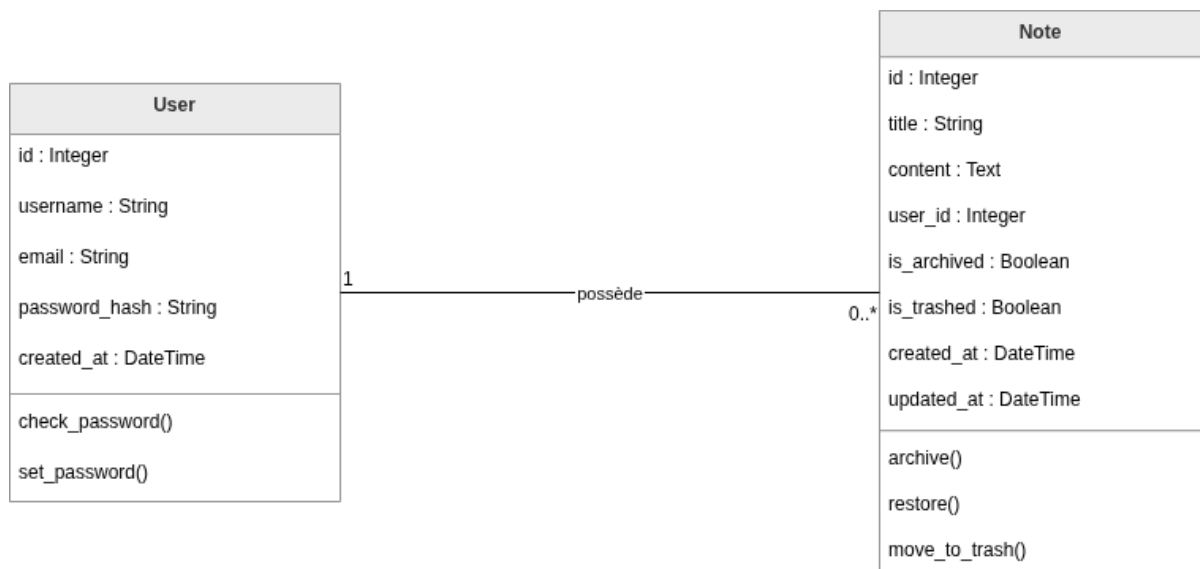


Fig. 5. – Diagramme de classes (modèle de données).

18.1. Description

- Un **User** possède de zéro à n **Notes**.
- Une **Note** appartient à exactement un **User**, via la clé étrangère `user_id`.
- Les drapeaux `is_archived` et `is_trashed` permettent une suppression *douce* (corbeille) et une archive sans déplacer ni dupliquer la donnée.

18.2. Considérations de sécurité dans le modèle

- Le champ `password_hash` contient uniquement le *hachage* du mot de passe, jamais sa valeur en clair.
- Les contraintes `UNIQUE` sur `username` et `email` préviennent les doublons et certaines attaques d'énumération.
- Toutes les requêtes `SELECT`, `UPDATE` ou `DELETE` sur la table `notes` **doivent** comporter le filtre `WHERE user_id = current_user.id`.

19. Diagrammes de séquence

19.1. Authentification

Le scénario d'authentification met en jeu cinq acteurs : l'utilisateur, son navigateur, Nginx, Flask [1] et PostgreSQL. La Fig. 6 en détaille le déroulement, depuis la saisie des identifiants jusqu'à la création de la session côté serveur et au positionnement du *cookie* dans le navigateur.

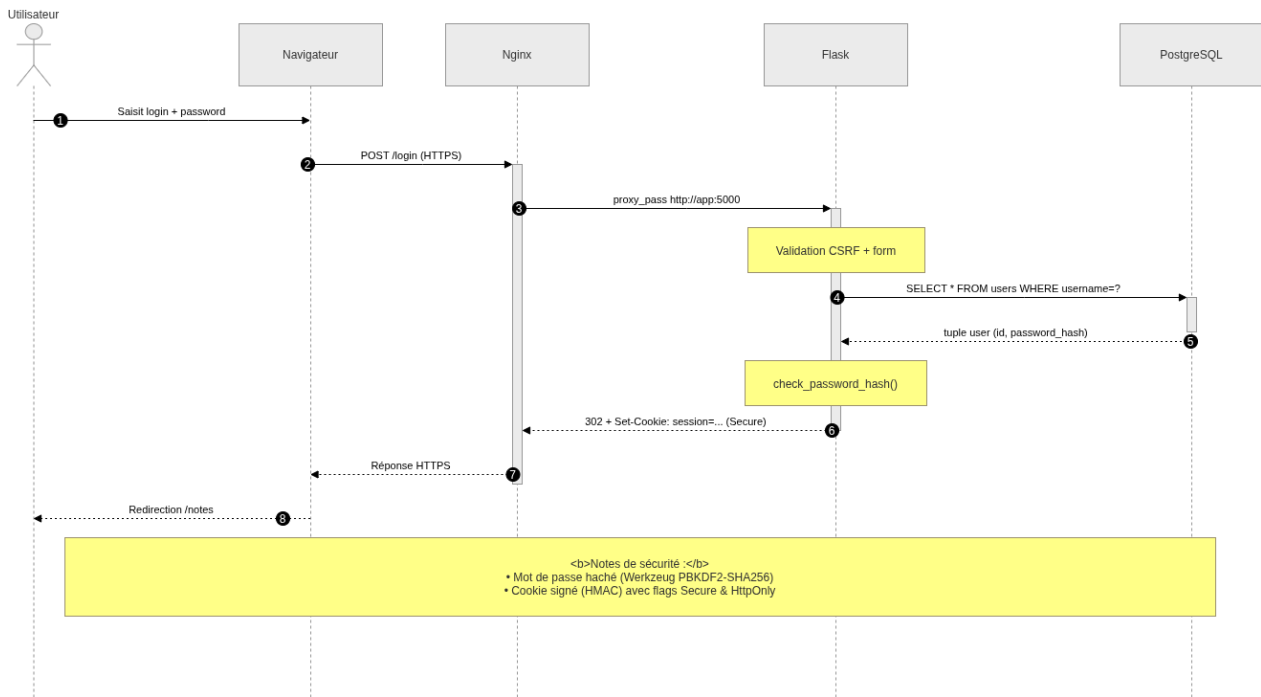


Fig. 6. – Diagramme de séquence --- authentification.

Trois éléments de sécurité sont à noter :

1. le mot de passe ne quitte jamais le tunnel TLS;
2. la comparaison côté serveur passe par `check_password_hash`, qui exécute une comparaison *constant-time*;
3. le *cookie* de session est signé (HMAC), et porte les attributs `Secure`, `HttpOnly` et `SameSite=Lax`.

19.2. Création d'une note

Le scénario de création d'une note (Fig. 7) montre en particulier le contrôle de session (`@login_required`), la vérification du jeton CSRF, puis l'insertion en base avec association à l'utilisateur courant.

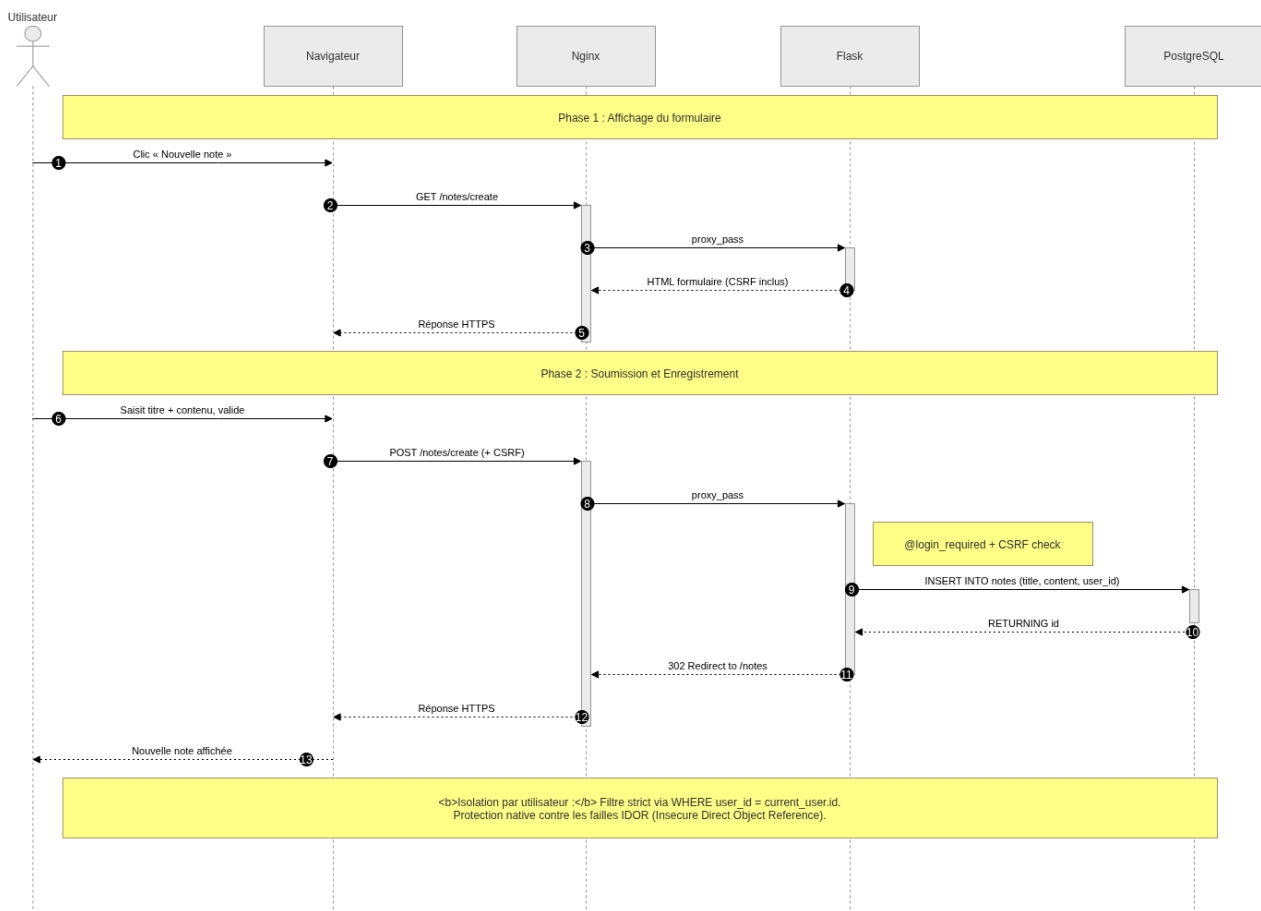


Fig. 7. – Diagramme de séquence --- création d'une note.

20. Conception de la sécurité — défense en profondeur

La sécurité du système ne repose pas sur une unique mesure, mais sur **six couches empilées**, chacune apportant une garantie spécifique. La Fig. 8 en donne une représentation synthétique.

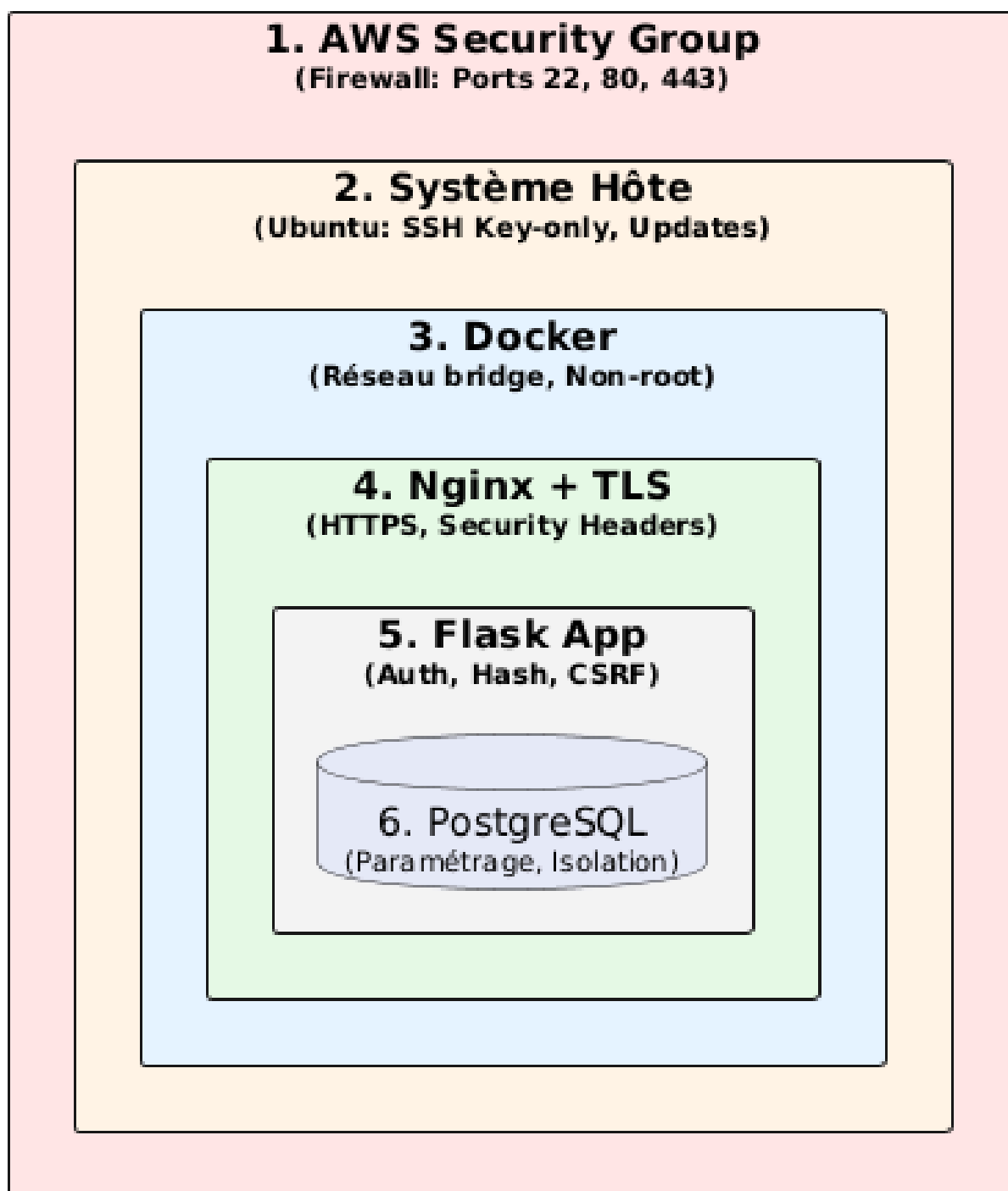


Fig. 8. – Les six couches de sécurité (_defense-in-depth_).

20.1. Couche 1 — Security Group

Le Security Group AWS est un pare-feu *stateful* qui constitue la **première ligne de défense** : il bloque tout port non explicitement autorisé. Sa configuration a été détaillée plus haut.

20.2. Couche 2 — Système hôte

Sur l'instance EC2 :

- `PermitRootLogin no` : interdiction du login SSH root;
- authentification SSH par clé en mode primaire;
- `unattended-upgrades` pour appliquer les mises à jour de sécurité automatiquement;

- utilisateur applicatif ubuntu non-root.

20.3. Couche 3 — Isolation Docker

- Réseau *bridge* privé webcyber-net; les conteneurs app et db ne publient pas de port;
- Le **Dockerfile** de l'application crée un utilisateur non-privilégié appuser (UID 1000) et l'utilise pour l'exécution finale;
- Les volumes nommés isolent les données persistantes du système de fichiers de l'hôte.

20.4. Couche 4 — Nginx + TLS

- TLS 1.2 et 1.3 uniquement, profil *Mozilla Intermediate*;
- certificat **Let's Encrypt** renouvelé automatiquement;
- redirection 80 → 443;
- en-têtes **HSTS**, **CSP**, **X-Frame-Options**, **X-Content-Type-Options**, **Referrer-Policy** injectés sur toutes les réponses;
- `server_tokens off` pour masquer la version de Nginx.

20.5. Couche 5 — Application Flask

- Hachage des mots de passe via `generate_password_hash` (PBKDF2-SHA256, sel automatique);
- Session signée par `SECRET_KEY` aléatoire (32 octets);
- Protection **CSRF** sur tous les formulaires (Flask-WTF);
- Validation systématique des entrées par WTForms (longueur, type, présence);
- Décorateur `@login_required` sur toutes les routes privées;
- Filtre `user_id` sur 100 % des requêtes *notes*.

20.6. Couche 6 — Base de données

- Utilisateur webcyber_user non-superutilisateur;
- Mot de passe long et aléatoire (>=20 caractères) injecté via `.env`;
- Pas de port publié vers l'hôte;
- Requêtes paramétrées par SQLAlchemy (pas de concaténation de chaînes SQL).

21. Conclusion du chapitre

La conception ainsi décrite est volontairement *minimale mais complète* : un seul serveur, trois conteneurs, deux entités, mais une sécurité empilée sur six niveaux. Le chapitre suivant présente la réalisation concrète de cette conception et les étapes de son déploiement.

Réalisation et déploiement

22. Introduction

Ce chapitre décrit la réalisation concrète de l'application WebCyber : choix technologiques détaillés, construction des images Docker, configuration de l'instance EC2, mise en place du DNS via Route 53, configuration de Nginx avec HTTPS Let's Encrypt, et procédure de déploiement *from scratch*. Il se termine par le diagramme de déploiement UML qui synthétise l'ensemble.

23. Environnement et outils

23.1. Poste de développement

- OS : Linux (Ubuntu 24.04);
- IDE : Visual Studio Code;
- Outils en ligne de commande : `git`, `docker`, `docker compose`, `terraform`, `tfsec`, `ssh`, `curl`, `dig`;
- Gestionnaire de versions : Git, dépôt distant sur GitHub.

23.2. Stack technologique retenue

Couche	Choix	Justification
	Cloud / IaaS	AWS EC2
Standard du marché, <i>free tier</i> , riche écosystème.	Système	Ubuntu 24.04 LTS
LTS, large support Docker.	IaC	Terraform 1.12 + provider AWS 5.98
Provisionnement déclaratif et reproductible de l'infra.	Conteneurs	Docker + Compose
Reproductibilité, isolation.	Registre images	GitHub Container Registry (GHCR)
Distribution de l'image applicative.	CI/CD	GitHub Actions
Pipeline build / scan / déploiement automatisé.	Sécurité IaC	tfsec (Aqua)
Analyse statique du code Terraform (SAST infra).	Proxy / TLS	Nginx 1.25
Léger, robuste, terminaison TLS éprouvée.	Langage	Python 3.11
Productivité, éco-système mature.	Framework web	Flask 3
Léger, transparent, pédagogique.	WSGI	Gunicorn
WSGI de production, multi- <i>worker</i> .	ORM	SQLAlchemy 2
Empêche par construction l'injection SQL.	Auth	Flask-Login
Gestion de session simple et bien testée.	Formulaires	Flask-WTF
Validation (+) CSRF intégrée.	Hash mot de passe	Werkzeug PBKDF2-SHA256
Sel automatique, coût configurable.	Base de données	PostgreSQL 16
Robuste, indexation et JSON natifs.	TLS / CA	Let's Encrypt + Certbot
Certificats gratuits et automatiques.	Front-end	Tailwind CSS
Mise en page rapide et responsive.		

Tableau 7. – Synthèse de la pile technologique.

24. Implémentation de l'application Flask

24.1. Organisation du code

Le code source de l'application est structuré selon le *pattern application factory* de Flask [1], qui sépare la création de l'application, les *blueprints* (groupes de routes) et les modèles de données.

24.2. Modèles SQLAlchemy

24.3. Authentification et isolation

Le `filter_by(user_id=current_user.id)` sur *toutes* les requêtes garantit qu'un utilisateur ne peut pas accéder aux notes d'un autre, même en forgeant un identifiant dans l'URL (parade contre l'IDOR).

25. Conteneurisation

25.1. Dockerfile applicatif

Trois bonnes pratiques sont appliquées : (i) image *slim* pour réduire la surface d'attaque, (ii) création d'un utilisateur `appuser` non-root, (iii) Gunicorn comme serveur WSGI au lieu du serveur de développement de Flask [1].

25.2. Fichier `docker-compose.yml`

26. Infrastructure as Code avec Terraform

26.1. Motivation

Plutôt que de provisionner l'infrastructure manuellement via la console AWS — démarche peu reproductible et sujette à l'erreur humaine —, l'infrastructure est décrite **en code** avec **Terraform** 5.98. Cette approche, dite *Infrastructure as Code (IaC)*, présente plusieurs avantages :

- **Reproductibilité** : la même configuration peut être rejouée à l'identique, en local comme en CI/CD;
- **Versionnement** : le code Terraform est versionné dans Git aux côtés du code applicatif;
- **Auditabilité** : chaque modification d'infrastructure passe par une *pull request* et un plan d'exécution (`terraform plan`) lisible en revue de code;
- **Analyse statique** : des outils comme `tfsec` peuvent inspecter le code et détecter les mauvaises configurations *avant* le déploiement (cf. chapitre 5).

26.2. Organisation des fichiers

26.3. Bloc Terraform et provider

26.4. Ressources principales

Le fichier `main.tf` décrit trois ressources et une *data source* :

1. `data "aws_ami" "ubuntu"` : recherche dynamique de la dernière AMI Ubuntu 24.04 LTS publiée par Canonical;
2. `aws_security_group "ssh"` : règles entrantes 22, 80, 443 ouvertes au monde, sortantes libres;
3. `aws_instance "instance"` : EC2 t3.micro, IMDSv2 forcé, *provisioner* `remote-exec` qui installe Docker (+) `Compose` au premier `apply`;
4. `aws_eip_association "instance"` : association de l'Elastic IP existante (`eipalloc-...`) à la nouvelle instance, afin que le DNS reste valide.

26.5. Cycle de provisionnement

À la fin de l'`apply`, Terraform affiche les sorties (*outputs*) :

L'EIP étant associée à l'instance, l'adresse publique reste stable malgré les redémarrages, ce qui maintient la cohérence des enregistrements DNS de Route 53.

27. Configuration DNS via Route 53

Sur Route 53, une zone hébergée publique est créée pour `webcyber.app` et deux enregistrements de type A sont ajoutés :

Nom	Type	Valeur	TTL
	<code>webcyber.app</code>	A	« <Elastic IP>
300	<code>www.webcyber.app</code>	A	<Elastic IP>
300			

Tableau 8. – Enregistrements DNS de la zone hébergée `webcyber.app`.

Le TLD `.app` étant inscrit sur la liste *HSTS preload*, le navigateur **refuse toute connexion non-HTTPS** : la mise en place de Let's Encrypt est donc obligatoire.

28. Préparation automatique de l'instance

À la différence d'une approche manuelle (`ssh`, `apt install`, configuration ligne par ligne), la préparation de l'EC2 est entièrement automatisée par deux mécanismes complémentaires :

1. Le *provisioner* remote-exec de Terraform installe Docker et active son service au moment du `terraform apply` (cf. extrait de `main.tf` ci-dessus);
2. Le pipeline CI/CD (section suivante) clone le dépôt, écrit le fichier `.env` à partir d'un *secret* GitHub, et démarre la pile Docker Compose.

Le fichier `.env` est ainsi **généré sur le serveur** à partir d'un secret stocké dans GitHub Actions et **n'est jamais versionné**; il est exclu par `.gitignore`.

29. Configuration Nginx et HTTPS

29.1. Configuration Nginx

Le proxy Nginx est configuré en deux blocs :

29.1.1. Détail des en-têtes.

En-tête	Rôle
	Strict-Transport-Security
Force le navigateur à utiliser HTTPS pendant un an (HSTS).	X-Content-Type-Options
Empêche le <i>MIME-sniffing</i> (nosniff).	X-Frame-Options
Bloque l'inclusion en <code><iframe></code> sauf en <i>same-origin</i> (anti-clickjacking).	Referrer-Policy
Limite la fuite d'URL via le <i>referer</i> .	Content-Security-Policy
Restreint les sources de scripts, styles, images, polices, ... (défense contre XSS).	server_tokens off
N'affiche pas la version de Nginx dans les en-têtes (réduction de la divulgation d'information).	

Tableau 9. – En-têtes de sécurité HTTP appliqués par Nginx.

29.2. Durcissement des cookies de session Flask

En complément des en-têtes HTTP, la configuration Flask [1] (app/config.py) durcit les attributs des cookies de session :

Ces trois attributs (Secure, HttpOnly, SameSite=Lax) sont effectivement présents dans la réponse HTTP, ce qui sera vérifié au chapitre 5.

29.3. Obtention du certificat Let's Encrypt

L'obtention du certificat suit la méthode *webroot* :

Une tâche cron quotidienne est ensuite ajoutée pour le renouvellement automatique :

29.4. Démarrage de la pile

En production, la pile est démarrée par le pipeline CI/CD (section suivante). En local, les commandes suivantes suffisent :

30. Pipeline CI/CD avec GitHub Actions

30.1. Vue d'ensemble

L'intégration et le déploiement continus sont assurés par **GitHub Actions**. À chaque *push* sur la branche *main*, un *workflow* en trois *jobs* séquentiels est déclenché :

1. **infra-scan** : analyse statique du code Terraform avec *tfsec*; un échec bloque la suite du pipeline;
2. **build-and-push** : construction de l'image Docker applicative et publication sur le *GitHub Container Registry* (GHCR);
3. **deploy** : connexion SSH à l'EC2, récupération du code, génération du fichier `.env` depuis un *secret*, puis `docker compose up`.

#	Job	Rôle
1		infra-scan
2	Scan tf-sec sur le dossier terraform/ (gate sécurité IaC).	build-and-push
3	docker build + push vers GH-CR avec tags latest et sha. SSH vers l'EC2, docker compose pull && up, gestion du certificat Let's Encrypt au premier déploiement.	deploy

Tableau 10. – Étapes du pipeline GitHub Actions.

30.2. Extrait du *workflow*

30.3. Secrets GitHub utilisés

Nom du secret	Contenu
	EC2_HOST
Adresse IP publique (Elastic IP) de l'instance.	EC2_USER
ubuntu (utilisateur SSH par défaut).	EC2_SSH_KEY
Contenu de la clé privée vockey.pem.	ENV_FILE
Contenu complet du fichier .env (clé secrète Flask [1], identifiants PostgreSQL, etc.).	

Tableau 11. – Secrets configurés dans le dépôt GitHub.

30.4. Bénéfices de l'approche

- **Boucle de retour rapide** : un *push* suffit pour livrer l'application en production en quelques minutes.
- **Séparation des responsabilités** : Terraform gère l'*infrastructure*, GitHub Actions gère l'*application*.
- **Sécurité par construction** : aucun commit contenant une mauvaise configuration Terraform de niveau *HIGH* ou *CRITICAL* ne peut atteindre la production, grâce au *gate tfsec*.
- **Aucun secret dans le dépôt** : le fichier .env et la clé SSH sont stockés exclusivement dans les *GitHub Secrets*.

31. Vue d'ensemble du flux HTTPS

La Fig. 9 reprend, sous forme synthétique, le trajet d'une requête HTTPS depuis le navigateur jusqu'à la base de données, en distinguant clairement la portion *chiffrée* (entre le client et Nginx) et la portion *en clair sur réseau Docker isolé* (entre Nginx, Flask [1] et PostgreSQL).

Figure 01 — Architecture globale

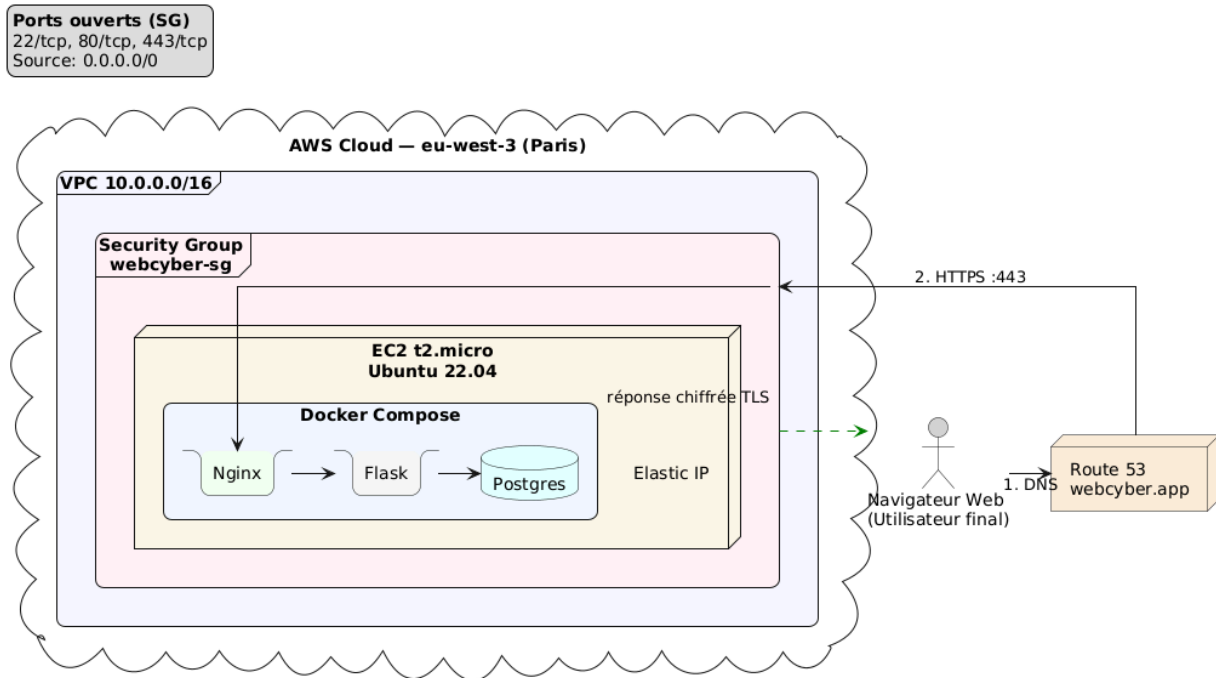


Fig. 9. – Flux d'une requête HTTPS, des en-têtes au stockage.

31.1. Captures d'écran attendues

La documentation prévoit l'insertion de captures d'écran illustrant :

- l'interface de connexion / inscription ;
- le tableau de bord de lecture et de création de notes ;
- le résultat du déploiement Docker Compose avec l'état des conteneurs.

Capture d'écran 1 : interface de connexion / inscription

Fig. 10. – Espace réservé pour une capture d'écran du formulaire de connexion.

Capture d'écran 2 : interface de gestion des notes

Fig. 11. – Espace réservé pour une capture d'écran de l'interface de notes.

32. Diagramme de déploiement

Le diagramme de déploiement UML (Fig. 12) résume les artefacts physiques et logiques mis en jeu, ainsi que les protocoles de communication entre eux.

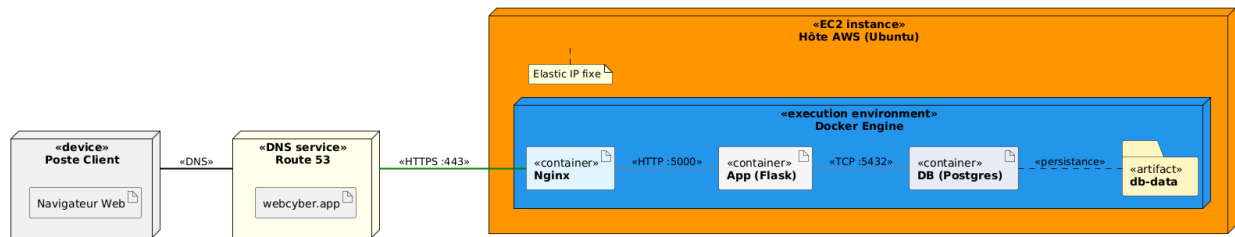


Fig. 12. – Diagramme de déploiement UML.

33. Conclusion du chapitre

La réalisation s'est appuyée sur des outils *standards de l'industrie* (Flask [1], PostgreSQL, Nginx, Docker, AWS, Let's Encrypt), ce qui a permis de concentrer l'effort sur l'**intégration sécurisée** plutôt que sur la maîtrise de technologies exotiques. Le résultat : un déploiement reproductible, conteneurisé, accessible en HTTPS sur `https://webcyber.app`, et préparé pour la phase de tests et d'audit qui fait l'objet du chapitre suivant.

Tests et analyse de sécurité

34. Introduction

Ce chapitre présente la dernière phase du projet : la **validation** fonctionnelle de l'application et l'**audit de sécurité** du déploiement. La méthodologie suit six phases complémentaires couvrant aussi bien le code applicatif que l'infrastructure : tests fonctionnels, analyse statique de l'IaC, vérification de l'infrastructure déployée, scan de ports, scan applicatif web, et évaluation de la configuration TLS.

35. Stratégie de test

Phase	Type	Outils	Objectif
—	1	Fonctionnel	Navigateur, curl
Vérifier les cas d'utilisation	2	Analyse statique IaC	tfsec
Détecter les défauts de configuration avant déploiement	3	Infrastructure	docker, dig, curl
Confirmer le déploiement	4	Scan de ports	Nmap
Limiter la surface d'attaque	5	Scan applicatif web	Nikto, curl
Détecter les mauvaises configurations HTTP	6	Évaluation TLS	SSL Labs, testssl.sh
Mesurer la qualité du chiffrement			

Tableau 12. – Phases de la campagne de validation.

36. Tests fonctionnels

36.1. Inscription et authentification

Les tests d'inscription et d'authentification couvrent à la fois les chemins nominaux (*happy path*) et les chemins d'erreur :

#	Cas	Entrée	Résultat attendu
— 1		Inscription valide	user/email/mdp corrects
Compté2 créé, re- di- rec- tion / lo- gin		Username existant	username déjà pris
Er- 3 reur, pas de dou- blon		Mot de passe court	< 8 caractères
Er- 4 reur de va- lida- tion		Champs vides	blanc
Er- reur de va- lida- tion			

Tableau 13. – Tests d'inscription.

#	Cas	Entrée	Résultat attendu
—	1	Identifiants valides	user/mdp corrects
Re- di- rec- tion / notes	2	Mauvais mot de passe	mdp erroné
Mes- sage d'erreur	3	Utilisateur inconnu	username inexistant
Mes- sage d'erreur	4	Accès direct sans auth	GET /notes
Re- di- rec- tion / lo- gin	5	Déconnexion	clic « se déconnecter »
Ses- sion dé- truite, re- tour / lo- gin			

Tableau 14. – Tests d'authentification.

36.2. Gestion des notes (CRUD)

#	Cas	Résultat attendu
1		Création de note
2	Note dans la liste	Détail d'une note
3	Titre + contenu complet affiché	Édition
4	Contenu mis à jour, updated_at actualisé	Archivage
5	Disparaît de /notes, apparaît dans /archive	Mise à la corbeille
6	Disparaît, apparaît dans /trash	Restauration
7	Revient dans la liste principale	Suppression définitive
8	Disparaît de la liste	Recherche par titre

Tableau 15. Les tests CRUD sur les notes.

36.3. Test critique : isolation des données

Ce test vérifie qu'un utilisateur ne peut pas accéder aux notes d'un autre.

#	Scénario	Résultat attendu
1	Utilisateur A crée la note id=42	
2	Utilisateur B se connecte	
3	B accède à /notes/42 directement	
4	B tente POST /notes/42/edit	

Tableau 16. – Tests d'isolation par utilisateur.

Résultat : l'application bloque toute tentative grâce au filtre `filter_by(user_id=current_user.id)` systématique combiné à `first_or_404()`.

37. Analyse statique de l'IaC avec tfsec

37.1. Principe et positionnement

tfsec est un *Static Application Security Testing* (SAST) spécialisé dans le code Terraform. Il examine le code *avant* tout déploiement et signale les défauts de configuration courants : ports ouverts trop largement, absence de chiffrement, métadonnées d'instance non protégées, etc.

Il est intégré au pipeline CI/CD (cf. [4]) en tant que premier *job* : si une règle de niveau *HIGH* ou *CRITICAL* est levée, la chaîne de *build* et de *deploy* est immédiatement interrompue. Cette logique garantit qu'aucune mauvaise configuration d'infrastructure n'atteint la production sans avoir été examinée.

37.2. Exécution

37.3. Résultats et décisions

L'analyse initiale a remonté trois constats. Chacun a fait l'objet d'une décision argumentée :

Règle	Sévé- rité	Description	Décision
	aws-ec2-no-public-ingress-sgr	HIGH	IMDSv2 non requis sur l'instance
Corrigée ajout du bloc <code>metadata_options { http_tokens "required" }</code>	: aws-ec2-enable-at-rest-encryption	HIGH	Volume <i>root</i> non chiffré
Acceptée et documentée jet académique sur AWS Academy, pas de données sensibles au repos; le chiffrement forcerait un remplacement de l'instance. Exclusion par <code>tfsec:ignore</code> avec justification.	aws-ec2-add-description-to-security-group	LOW	Le <i>security group</i> utilise la description par défaut
Acceptée et documentée : <i>finding</i> cosmétique de bas niveau; ajouter une description force le remplacement du SG. Exclusion par <code>tfsec:ignore</code> .			

Tableau 17. – Constats tfsec sur le module Terraform et décisions associées.

Les deux exclusions sont matérialisées par des commentaires `tfsec:ignore` placés directement au-dessus des ressources concernées, accompagnés d'un libellé expliquant le motif. Ce procédé laisse une trace explicite et auditable des risques acceptés, ce qui est préférable à la suppression silencieuse via une option de la ligne de commande.

37.4. Résultat final

Après correction d'IMDSv2 et exclusion documentée des deux *findings* restants, le scan passe sans alerte :

Le pipeline CI/CD passe désormais le *gate* infra-scan sur chaque push, et tout futur ajout de mauvaise configuration sera bloqué automatiquement.

38. Vérification de l'infrastructure

38.1. Résolution DNS

Les deux enregistrements pointent bien vers l'Elastic IP de l'instance EC2.

38.2. État des conteneurs

38.3. Connectivité

La redirection HTTP → HTTPS fonctionne et le service HTTPS répond en HTTP/2.

38.4. Captures d'écran attendues

Des captures d'écran faciliteront l'illustration des résultats de la phase de vérification :

- sortie de `docker compose ps` ;
- réponse HTTPS valide dans le navigateur ;
- exemples de pages publiques et privées de l'application.



Capture d'écran 3 : état des conteneurs Docker Compose

Fig. 13. – Espace réservé pour une capture d'écran de la pile Docker en cours d'exécution.

39. Scan de ports avec Nmap

39.1. Objectif

Nmap permet de **vérifier de l'extérieur** quels ports sont accessibles sur la cible. L'objectif est de confirmer que le *Security Group* laisse uniquement passer ce qui est attendu, et que les services internes (Flask [1], PostgreSQL) sont bien invisibles.

39.2. Commandes utilisées

39.3. Résultats observés

Port	État	Service	Analyse
—	22	open	OpenSSH 8.x
At- ten- du : ad- mi- nis- tration.	80	open	Nginx 1.25 (redirect)
At- ten- du : re- di- rec- tion 301 vers HTTPS.	443	open	Nginx 1.25 (SSL)
At- ten- du : tra- fic web.	5000	filtered	—
Cor- rect : Flask non ex- po- sé.	5432	filtered	—
Cor- rect : Post- greS- QL non ex- po- sé.	Tous les autres	filtered	—
Cor- rect : sur- face d'attaque mi- nimale.			

Tableau 18. – Résultats du scan Nmap.

Conclusion du scan : la surface d'attaque réseau est conforme à la conception. Aucun service interne n'est joignable depuis Internet.

39.4. Recommandations

- **Masquer la version** de Nginx avec `server_tokens off` (déjà appliqué).
- En production réelle, restreindre la source du port 22 à une plage d'IP de confiance plutôt que `0.0.0.0/0`.

40. Scan applicatif Web avec Nikto

40.1. Objectif

Nikto est un scanneur *open-source* qui contrôle la présence de mauvaises configurations courantes : entêtes manquants, fichiers oubliés, divulgation d'informations, scripts par défaut, etc.

40.2. Commande

40.3. Résultats typiques et interprétation

Constat	Sévérité	Mitigation
	HSTS présent	Info
Attendu (.app <i>HSTS-preload</i>).	Header CSP default-src 'self' présent	Info
Configuré dans Nginx.	X-Frame-Options: DENY présent	Info
Configuré dans Nginx.	X-Content-Type-Options: nosniff présent	Info
Configuré dans Nginx.	Aucun fichier sensible (.git, .env, etc.) accessible	—
Attendu : pas de fichiers exposés.	Pas de version Nginx visible	Info
server_tokens off.	Pas d'injection SQL ou XSS détectée par Nikto	Info
ORM SQLAlchemy + Jinja2 auto-escape.		

Tableau 19. – Synthèse des résultats Nikto.

Conclusion : aucune vulnérabilité *critique* ou *haute* n'a été remontée. Les en-têtes de sécurité sont tous présents, les chemins sensibles sont fermés, et la version du serveur n'est pas divulguée.

41. Évaluation TLS

41.1. SSL Labs

L'outil *SSL Labs Server Test* de Qualys (<https://www.ssllabs.com/ssltest/>) a été utilisé pour évaluer la configuration TLS du domaine.

Note attendue : A ou A+.

41.2. testssl.sh

41.3. Résultats

Critère	Résultat attendu
	Note SSL Labs
A ou A+	Protocoles
TLS 1.2 et 1.3 uniquement	TLS 1.0 / 1.1 / SSLv3
désactivés	Suites de chiffrement
profil Mozilla <i>Intermediate</i>	Certificat
Let's Encrypt valide, chaîne complète	<i>Forward Secrecy</i>
oui (ECDHE)	HSTS
présent (max-age=31536000)	OCSP Stapling
activé	Vulnérabilités connues (Heartbleed, POODLE...)
non détectées	

Tableau 20. – Synthèse de l'évaluation TLS.

42. Synthèse des audits

Domaine	Statut	Commentaire
	Sécurité de l'IaC (tfsec)	OK
0 <i>finding</i> actif, 2 exclusions documentées	IMDSv2 forcé sur l'EC2	OK
http_tokens = "required"	Surface réseau (Nmap)	OK
Seuls 22, 80, 443 ouverts	En-têtes HTTP (Nikto)	OK
HSTS, CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy	Cookies de session	OK
Secure, HttpOnly, SameSite=Lax	Configuration TLS	OK
TLS 1.2/1.3, FS, HSTS	Authentification	OK
PBKDF2-SHA256, sessions signées	Protection CSRF	OK
Token Flask-WTF sur tous les POST	Protection injection SQL	OK
ORM paramétré	Protection XSS	OK
Jinja2 auto-escape (+) restrictif	Isolation par utilisateur	OK
Filtre user_id systématique	Secrets	OK
.env hors Git, géré via <i>GitHub Secrets</i>	Pipeline CI/CD	OK
<i>Gate</i> tfsec bloquant en amont du <i>build</i>		

Tableau 21. – Tableau de bord de sécurité.

43. Limites de la campagne

- Les outils utilisés (Nmap, Nikto) sont des scanners *boîte noire* : ils ne remplacent pas un audit manuel ni un test d'intrusion approfondi.
- Aucune analyse statique de code (SAST) n'a été menée sur le code Python; l'utilisation d'outils tels que **bandit** ou **semgrep** constitue une perspective.
- L'application étant simple (CRUD textuel), le risque applicatif résiduel est faible; néanmoins une analyse formelle de modèle de menace (*threat modeling*) serait pertinente sur une application de plus grande envergure.

44. Conclusion du chapitre

La phase de tests confirme à la fois le bon fonctionnement *fonctionnel* de l'application et la *solidité* de son déploiement vis-à-vis des attaques courantes. Les principaux objectifs de sécurité fixés dans le chapitre 2 sont remplis : isolation réseau, HTTPS strict, en-têtes durcis, isolation des données, et hachage robuste des mots de passe.

Conclusion générale

Ce projet WebCyber a permis de concevoir, de réaliser et de déployer une application web sécurisée et conteneurisée, tout en respectant des objectifs clairs de qualité, de reproductibilité et de sécurité.

45. Bilan du projet

Les principaux objectifs ont été atteints :

- une application de prise de notes fonctionnelle, avec authentification, gestion CRUD, archive, corbeille et recherche;
- une architecture conteneurisée composée de Nginx, Flask [1] et PostgreSQL orchestrés par Docker Compose;
- un déploiement sur AWS avec EC2, Elastic IP, Security Group et Route 53;
- une chaîne HTTPS opérationnelle via Let's Encrypt, avec renouvellement automatisé;
- une approche de sécurité en profondeur fondée sur six couches : réseau, hôte, isolation Docker, proxy/TLS, application et base de données;
- une campagne de validation couvrant les tests fonctionnels et l'audit de sécurité (Nmap, Nikto, SSL Labs/testssl.sh).

Ce rapport montre que la solution développée est cohérente avec le périmètre académique du PFE tout en conservant des pratiques proches de l'industrie.

46. Apports méthodologiques

Le travail réalisé a apporté plusieurs acquis importants :

- maîtrise des mécanismes de conteneurisation et des flux de données entre services Docker;
- compréhension de la sécurisation d'un déploiement web en production : gestion des certificats, configuration de Nginx, règles de pare-feu, isolation des services;
- capacité à documenter une architecture technique complète, de la conception à l'implémentation et au test;
- adoption d'une démarche d'audit pragmatique, fondée sur des outils standard et sur des conclusions actionnables.

47. Limites et perspectives d'évolution

Bien que la solution soit robuste pour un usage pédagogique et de démonstration, plusieurs améliorations restent possibles :

- mise en place d'un pipeline CI/CD pour automatiser les tests et le déploiement;
- supervision et journalisation centralisées (Prometheus, Grafana, ELK/CloudWatch);
- audit de code statique (bandit, semgrep) et tests de sécurité automatisés plus poussés;
- renforcement de l'authentification par 2FA ou OAuth;
- migration vers une solution d'orchestration plus avancée (Kubernetes, ECS) pour améliorer la résilience et l'extensibilité;
- sauvegardes automatisées de la base PostgreSQL et procédure de restauration.

Ces pistes constituent des prolongements naturels pour transformer WebCyber en une plateforme encore plus professionnelle et adaptée à un contexte de production.

48. Conclusion

WebCyber illustre qu'un projet de fin d'études peut être à la fois simple dans son périmètre fonctionnel et ambitieux dans son approche architecturale et de sécurité. La démarche adoptée, centrée sur la reproductibilité et la défense en profondeur, offre une base solide pour évoluer vers des déploiements plus complexes et des applications plus critiques.

Bibliographie

- [1] Pallets Projects, « Flask Documentation ». [En ligne]. Disponible sur: <https://flask.palletsprojects.com/>
- [2] National Institute of Standards and Technology (NIST), « The NIST Definition of Cloud Computing », n° Special Publication 800-145. National Institute of Standards, Technology, 2011.
- [3] OWASP Foundation, « OWASP Top Ten 2021 ». [En ligne]. Disponible sur: <https://owasp.org/Top10/>
- [4] OWASP Foundation, « CI/CD Security Best Practices ». [En ligne]. Disponible sur: <https://owasp.org/www-project-ci-cd-security/>